

Interactive Curriculum Engine

Master Planning Document

v1.0 — Pre-implementation Blueprint

Project	Dynamic Video to Interactive Curriculum Engine
Domain	EdTech · Video Understanding · Multimodal AI · Curriculum Generation
Team	Zubair (Full-stack) · Aryan (AI/NLP/CV lead) · Muhammad Ahmed (CV/Hardware)
Version	v1.0 — decisions locked
Status	Single source of truth — to be refined as R&D completes
Date	July 2026

*This document is the foundational blueprint for a 3-person startup project.
It covers architecture, roles, models, research, phases, and evaluation.*

Contents

Locked Decisions (Ratified)	4
1 Project Understanding & Scope	5
1.1 Core Problem	5
1.2 Value Proposition	5
1.3 Target Users	5
1.4 MVP Scope (In)	5
1.5 Stretch Goals (Out of MVP, Phase 6+)	6
1.6 Key Technical Challenges & Open Decisions	6
2 Comprehensive Use Cases & User Stories	7
2.1 Primary Use Cases	7
2.1.1 UC-1 · Submit video for ingestion — (L)	7
2.1.2 UC-2 · Generate interactive curriculum — (S, internal)	7
2.1.3 UC-3 · Start interactive learning session — (L)	7
2.1.4 UC-4 · Encounter checkpoint exercise — (L, S)	7
2.1.5 UC-5 · Answer MCQ — (L, S)	7
2.1.6 UC-6 · Solve coding challenge — (L, S)	7
2.1.7 UC-7 · Debug a broken snippet — (L, S)	7
2.1.8 UC-8 · Answer conceptual question — (L, S)	8
2.1.9 UC-9 · Evaluation & feedback — (S)	8
2.1.10 UC-10 · Adaptive progression — (S)	8
2.1.11 UC-11 · Progress dashboard — (L)	8
2.1.12 UC-12 · Concept explanation viewer — (L)	8
2.1.13 UC-13 · Retry / revisit — (L)	8
2.1.14 UC-14 · Instructor reviews generated curriculum — (I)	8
2.1.15 UC-15 · Content library management — (I)	8
2.2 Secondary Use Cases	8
2.3 Edge-Case Use Cases (mitigations in §3)	8
3 Edge Cases & Risk Analysis	10
4 System Architecture & Module Breakdown	11
4.1 High-Level Architecture (text diagram)	11
4.2 Module Specifications	11
4.2.1 M1 · Ingestion Pipeline — (Lead: Zubair; Support: Ahmed)	11
4.2.2 M2 · Transcript & Metadata Extraction — (Lead: Aryan; Support: Zubair)	12
4.2.3 M3 · Visual Content Extraction — (Lead: Ahmed; Support: Aryan)	12
4.2.4 M4 · Lesson Structure Analyzer — (Lead: Aryan; Support: Ahmed)	12
4.2.5 M5 · Knowledge Graph / Concept Mapper — (Lead: Aryan; Support: Zubair)	12
4.2.6 M6 · Checkpoint Placement Controller — (Lead: Aryan; Support: Zubair)	13
4.2.7 M7 · Exercise Generation Engine — (Lead: Aryan; Support: Ahmed for code-context)	13
4.2.8 M8 · Test Generation & Validation — (Lead: Aryan; Support: Zubair for sandbox)	13
4.2.9 M9 · Answer Evaluation & Feedback Engine — (Lead: Aryan; Support: Zubair)	13
4.2.10 M10 · Adaptive Progression Controller — (Lead: Aryan; Support: Zubair)	13
4.2.11 M11 · Learner Profile & Progress Tracker — (Lead: Zubair; Support: Aryan)	14
4.2.12 M12 · Frontend Application — (Lead: Zubair; Support: Aryan for eval data shapes)	14

4.2.13	M13 · Backend API & Database — (Lead: Zubair)	14
4.2.14	M14 · Code Execution Sandbox — (Lead: Zubair; Support: Aryan)	14
4.2.15	M15 · Evaluation & Monitoring Dashboard — (Lead: Zubair; Support: Aryan for metrics)	14
5	Team Role Definition & Ownership	15
5.1	Responsibility Matrix (RACI)	15
5.2	Extent of Involvement	15
5.2.1	Zubair (Full-stack / System Glue) — owns product surface & reliability	15
5.2.2	Aryan (AI Lead) — owns the intelligence & correctness	15
5.2.3	Muhammad Ahmed (CV/Hardware) — owns perception & multimodal fusion input	15
5.3	Interface Contracts (must be agreed before coding)	16
5.3.1	Shared data formats (canonical JSON)	16
5.3.2	AI service API surface (Aryan owns, Zubair consumes)	16
5.3.3	Storage schema (Zubair owns)	16
6	Technology Stack & Model Requirements	17
6.1	AI / Model Layer — (Aryan + Ahmed)	17
6.2	Application / Infra Layer — (Zubair)	17
6.3	Hardware / Infrastructure	18
6.4	Fallback / Degradation Strategy	18
7	Pre-Implementation Research & Reading List	19
7.1	NLP / Transcript Understanding — (Aryan)	19
7.2	Question Generation — (Aryan)	19
7.3	Code Generation & Evaluation — (Aryan)	19
7.4	Vision / OCR / Screen Understanding — (Ahmed)	19
7.5	Multimodal / Video-Language — (Aryan + Ahmed)	20
7.6	Adaptive Learning — (Aryan)	20
7.7	System Design / Infra — (Zubair)	20
8	Development Phases & Milestones	21
8.1	Phase 0 · R&D Spike & Foundations — (Week 1) — All	21
8.2	Phase 1 · Ingestion + Transcript + Vision Extraction — (Weeks 2–3)	21
8.3	Phase 2 · Concept Segmentation + Knowledge Graph + Checkpoints — (Weeks 4–5)	21
8.4	Phase 3 · Exercise Generation + Test Validation + Evaluation — (Weeks 6–8)	21
8.5	Phase 4 · Interactive Frontend + Adaptive Logic + Progress — (Weeks 9–11)	21
8.6	Phase 5 · Polish, Evaluation, Deployment — (Weeks 12–13) — All	22
8.7	Phase 6 · Stretch Features — (post-MVP) — All	22
9	Documentation & Guidelines Blueprint	23
10	Innovation & Creative Ideas	24
11	Final Checklist Before Implementation Start	25
11.1	R&D	25
11.2	Environments & Access	25
11.3	Contracts & Schemas	25
11.4	Quality gates	25
A	Data Schema (ER)	26

B	AI Service API Contracts (OpenAPI sketch)	27
C	Prompt Library Spec	29
	C.0.1 Directory layout:	29
	C.0.2 manifest.yaml per prompt:	29
D	Golden Test Set & Eval Rubrics	30
	D.1 Golden test set (5 videos)	30
	D.2 Segmentation rubric (Aryan)	30
	D.3 Exercise quality rubric (human-rated, Aryan)	30
	D.4 Anti-cheat FP rate (Aryan)	30
	D.5 Eval-engine agreement (Aryan)	30
	D.6 System metrics (Zubair)	30

Locked Decisions (Ratified)

1. **LLM Strategy** → **Hybrid**. GPT-4o for high-value generation, grading, and structured reasoning; Llama 3.1 70B / Qwen2.5-Coder for bulk tasks, fallback, and sovereignty path.
2. **Code Sandbox** → **judge0 (MVP)** → **Firecracker microVM (prod)**.
3. **MVP Languages** → **Python only** for coding exercises. JS/TS deferred to Phase 6.
4. **Video Sources** → **YouTube URL (yt-dlp)** + **direct file upload**.
5. **Deployment** → **Multi-tenant cloud** with row-level data isolation + auth from day one.

Resolved: All locked decisions above. No further open decisions block Phase 0.

1 Project Understanding & Scope

1.1 Core Problem

Developers consume thousands of hours of free tutorials (YouTube, Loom, etc.), yet fall into “**tutorial hell**” — they can follow an instructor line-by-line but cannot reproduce the work independently. Watching $\neq$ learning because passive viewing rarely triggers independent reasoning. When learners eventually start their own projects, they remember syntax but not *when* or *why* to apply it.

Meanwhile, building interactive programming curricula manually (exercises, test cases, updates) is enormously expensive and decays as technologies evolve. Teams of instructors must design curricula, create exercises, prepare automated test cases, and continuously update content.

1.2 Value Proposition

An AI platform that **converts any technical tutorial video into a structured, interactive learning session** — automatically generating checkpoints, MCQs, coding challenges (in a *new context*, not the instructor’s example), debugging tasks, and conceptual questions that test *transfer of understanding*, not recall. The learner is forced to actively apply concepts before the video continues, and difficulty adapts to their performance.

Core value: Escaping tutorial hell by converting passive watching into active practice — at a fraction of the cost of manual curriculum creation, and reusing the vast existing body of free tutorial videos.

1.3 Target Users

- **Primary:** Self-taught developers, bootcamp students, junior engineers upskilling via YouTube tutorials.
- **Secondary:** Content creators/instructors who want their videos to become interactive courses; L&D teams curating internal video libraries.
- **Tertiary (future):** Non-programming technical learners (data science, math, electronics); collaborative/cohort learning.

1.4 MVP Scope (In)

Capability	MVP
Video ingestion via YouTube URL + direct file upload	✓
ASR transcript with word-level timestamps	✓
Concept segmentation + milestone/checkpoint placement	✓
Visual code OCR from IDE screenshots	✓
Exercise types: MCQ, coding challenge, debugging task, conceptual Q	✓
Auto test-case generation + sandboxed Python execution (judge0)	✓
Answer evaluation + feedback (incl. LLM-as-judge)	✓
Interactive video player with exercise overlays	✓
Per-session progress + simple adaptive difficulty (IRT 3PL / heuristic)	✓
Basic instructor/admin content dashboard	✓
API for curriculum generation	✓
Multi-tenant auth + row-level isolation	✓
MVP language: Python only for coding exercises	✓

1.5 Stretch Goals (Out of MVP, Phase 6+)

- Spaced repetition (FSRS-4.5), auto flashcards & notes.
- Concept dependency graph visualization; prerequisite detection.
- Project recommendations; interview-style timed questions.
- Code-review simulation (“PR review” exercises).
- Voice-interactive checkpoints (TTS/STT).
- Collaborative / multiplayer mode; cohort leaderboards.
- Multi-language content & multiple coding languages (JS/TS/Java/C++).
- Hardware companion device (ESP32 e-ink display, focus sensor) — Ahmed’s domain.
- Real-time on-device inference; edge/offline mode.

1.6 Key Technical Challenges & Open Decisions

1. **Concept segmentation quality** — where to cut topics and place checkpoints (no silver-bullet algorithm). Mitigation: hybrid signals (transcript pause + embedding shift + slide change) + LLM refine.
2. **Code OCR from IDE screenshots** — variable fonts, syntax highlighting, partial code, scrolling (the CodeSCAN problem).
3. **Generating correct, non-trivial, testable coding exercises** in a *different context* than the instructor’s (force transfer, not recall).
4. **Automated test generation + grading** — how to verify AI-generated tests are sound (self-consistency, mutation testing, execution-based filtering).
5. **Hallucination control** — factual errors, hallucinated APIs, outdated framework calls. Mitigation: cross-check vs extracted code + transcript entities; execution tests; LLM-judge self-consistency.
6. **Cost control** — LLM token costs across long videos. Mitigation: caching + tiered model usage + chunking + dedupe.
7. **Latency** — interactive UX requires pre-generation; the heavy pipeline must be async (Celery); session-time calls limited to fast evaluation.
8. **“When to interrupt”** — checkpoint cadence (too frequent = frustrating, too sparse = passive).
9. **Open-ended code grading** — beyond pass/fail tests: partial credit, style, edge-case handling (LLM-judge rubric).
10. **Evaluation of the system itself** — how do we know generated exercises are good? (human-rated golden set + benchmarks.)
11. **Multi-tenant isolation** — row-level security in Postgres; per-tenant object-storage prefixes; sandbox tenant tagging.
12. **Multi-tenancy + abuse** — per-tenant rate limits & token budgets; content moderation on submitted videos.

2 Comprehensive Use Cases & User Stories

Actors: Learner (L), Instructor/Admin (I), System/AI (S), Anonymous Visitor (V, future).

2.1 Primary Use Cases

2.1.1 UC-1 · Submit video for ingestion — (L)

- **Flow:** L pastes YouTube URL or uploads MP4 → backend validates (length, format, duplicate hash) → creates a `Curriculum` job (status `queued`) → returns `curriculum_id` → L is notified (poll/WebSocket) when ready.
- **Pre-conditions:** Authenticated; within tenant quota.
- **System response:** Async pipeline kicks off; ETA shown; progress events on WS.

2.1.2 UC-2 · Generate interactive curriculum — (S, internal)

- **Flow:** Pipeline runs: ingest → ASR → OCR → segment → extract concepts → place checkpoints → generate exercises + tests → validate → store → mark `ready`.
- **System response:** Structured curriculum JSON persisted; failures retried with backoff; partial results flagged; low-confidence items surfaced to admin.

2.1.3 UC-3 · Start interactive learning session — (L)

- **Flow:** L opens `curriculum_id` → loads player + checkpoint timeline → session state initialized → video plays from resume point.
- **System response:** Restore resume; render checkpoint markers on scrubber; prefetch next exercise.

2.1.4 UC-4 · Encounter checkpoint exercise — (L, S)

- **Flow:** Video auto-pauses at checkpoint timestamp → exercise modal overlays player → L attempts → submits.
- **System response:** Evaluate → feedback (verdict + explanation + hints) → on success, unlock resume; on fail, offer hint/retry/related-concept.

2.1.5 UC-5 · Answer MCQ — (L, S)

- Direct option match; distractor tags recorded; optional “explain your choice” to detect guessing.

2.1.6 UC-6 · Solve coding challenge — (L, S)

- **Flow:** Monaco editor loads with prompt + starter code + visible tests → L writes Python solution → “Run tests” → judge0 sandbox executes visible + hidden tests → results panel (per-test pass/fail, stdout diff, runtime, error trace) → LLM feedback on failure.
- **System response:** Verdict; partial credit; code-quality notes; anti-cheat similarity check vs instructor code.

2.1.7 UC-7 · Debug a broken snippet — (L, S)

- Buggy code presented; L fixes; hidden tests must pass; L must also submit a short explanation of the bug (LLM-graded).

2.1.8 UC-8 · Answer conceptual question — (L, S)

- Free-text → LLM-as-judge with rubric + reference answer; embedding similarity threshold for auto-pass; partial credit.

2.1.9 UC-9 · Evaluation & feedback — (S)

- Returns verdict, score, explanation, hints, anti-cheat flag, next recommended action.

2.1.10 UC-10 · Adaptive progression — (S)

- Performance vector updates skill model → next checkpoint difficulty recalculated → if mastery low, insert remedial exercise (regenerate simpler analog).

2.1.11 UC-11 · Progress dashboard — (L)

- Concepts mastered, accuracy streak, weak concepts, time spent, session history, “tutorial hell score.”

2.1.12 UC-12 · Concept explanation viewer — (L)

- Click any concept → auto-generated short explanation + source timestamp + related concept links.

2.1.13 UC-13 · Retry / revisit — (L)

- Re-attempt failed exercise; option to regenerate a fresh variant (different context, same concept).

2.1.14 UC-14 · Instructor reviews generated curriculum — (I)

- Dashboard lists curricula → per-checkpoint preview → approve / edit / regenerate → flag low-confidence items → publish/draft.

2.1.15 UC-15 · Content library management — (I)

- List, delete, tag, version, re-ingest updated video; per-tenant library.

2.2 Secondary Use Cases

- UC-16 Generate flashcards from a segment.
- UC-17 View concept dependency graph (stretch).
- UC-18 Export notes (Markdown/PDF).
- UC-19 Spaced-repetition reminders (stretch).
- UC-20 Re-ingest on video update.
- UC-21 Upload own code for AI review (stretch).
- UC-22 Interview-mode (timed, no hints).

2.3 Edge-Case Use Cases (mitigations in §3)

- UC-23 Unstructured video (interview, vlog — no clear topics).
- UC-24 Non-English audio.
- UC-25 Illegible code on screen.
- UC-26 Poor/no audio.
- UC-27 Multi-speaker / heavy digressions.

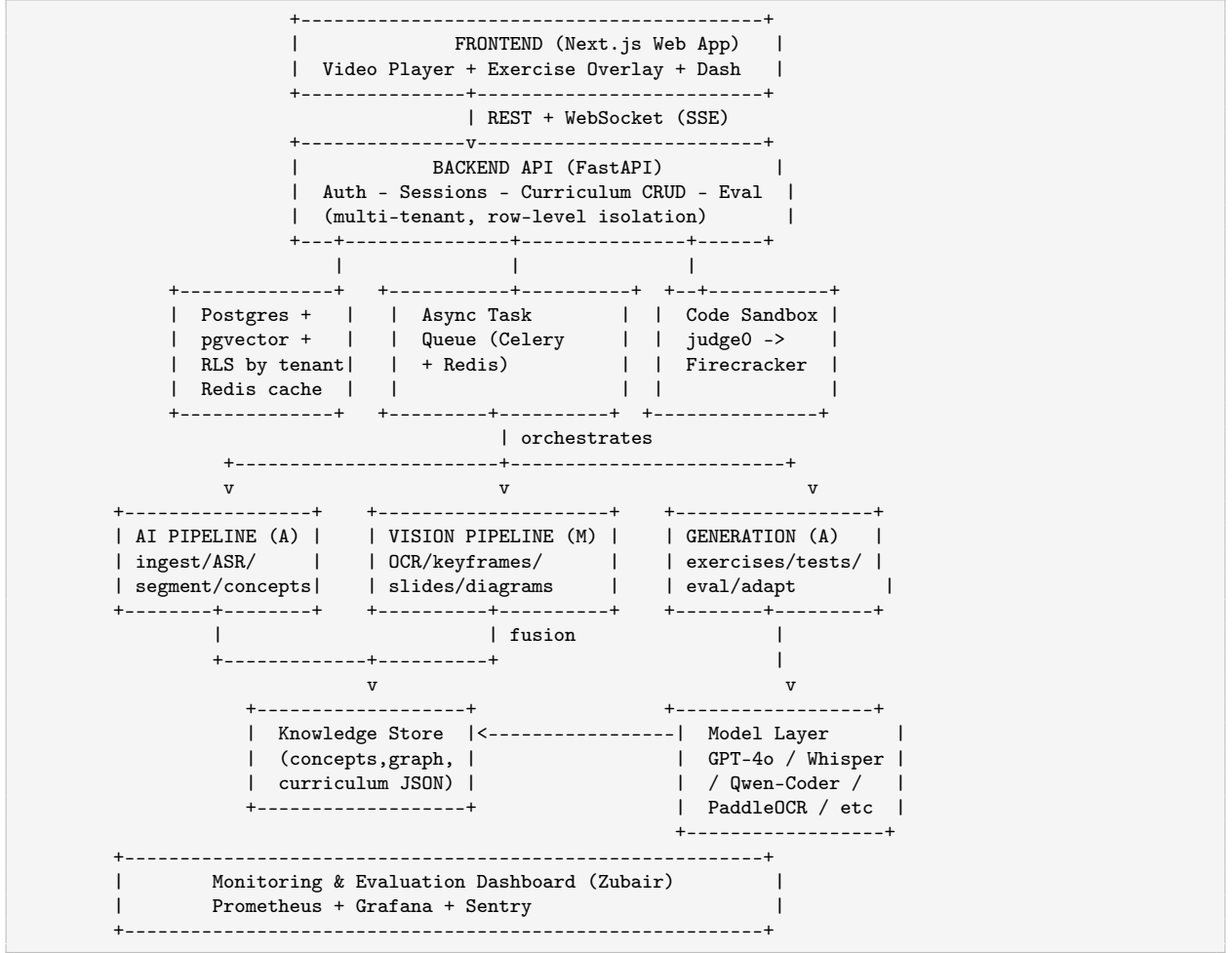
- UC-28 Very long (>1 h) / very short (<2 min) video.
- UC-29 Non-code technical video (math, electronics).
- UC-30 Conflicting/outdated tech (deprecated API).
- UC-31 Learner fails every checkpoint.
- UC-32 Cheating (pasting instructor's code, copying answers).
- UC-33 Video already has poor auto-captions (must override, not trust).
- UC-34 Video URL is private/age-restricted/deleted.
- UC-35 Uploaded file is corrupted/unsupported codec.

3 Edge Cases & Risk Analysis

#	Risk / Edge Case	Impact	Mitigation Strategy
E1	Low-res video / illegible code	OCR garbage → bad exercises	Confidence-score OCR; below threshold, fall back to transcript-only exercises; flag “code unclear”; upscale frames (Real-ESRGAN) pre-OCR.
E2	No transcript / poor audio	ASR failure	Whisper large-v3 (robust to noise); VAD skip silence; if WER high, degrade to slide+concept-only; allow manual transcript upload.
E3	Non-English content	Wrong segmentation	Detect language (Whisper auto); MVP supports English + code; others marked “experimental” with reduced guarantees.
E4	Unstructured video	No milestones	“Structuredness score” via LLM; if low, segment by time windows + topic; warn user.
E5	Multi-speaker / digressions	Topic noise	Speaker diarization (pyannote); relevance LLM filter to drop filler/off-topic segments.
E6	Very long videos (>1 h)	Cost + timeout	Chunk pipeline (5-min windows); parallelize; cache embeddings; tier model (cheap drafts, GPT-4o refines); per-curriculum token budget.
E7	AI hallucination (fake API, wrong method)	Learner misled	Cross-check generated code vs extracted code + transcript entities; execution tests; LLM-judge self-consistency (sample N, keep majority); version-tag framework.
E8	Outdated tech (old framework)	Misleading exercises	Detect framework version from transcript/screenshots; tag curriculum with version + date; warn “content from YYYY — verify against current docs.”
E9	Ambiguous topic boundaries	Over/under-segmentation	Multi-signal (transcript pause + slide change + topic embedding shift); merge adjacent same-topic segments; instructor can nudge boundaries.
E10	Beginner vs advanced mismatch	Frustration/boredom	Initial calibration question; per-concept difficulty tags; adaptive band.
E11	Multiple languages/frameworks in one video	Wrong test harness	MVP: Python only — detect non-Python and mark checkpoint as MCQ/conceptual instead of coding.
E12	Learner cheats (copies instructor code)	False mastery	Code-similarity detection (CodeBLEU/AST diff) vs instructor’s extracted code; if too similar, reject + regenerate new variant.
E13	Learner guesses MCQs	False skill signal	“Explain your choice” step; track response time; lower weight of guessed items.
E14	AI generates unsolvable coding challenge	Learner blocked	Validate every challenge by solving with a solver LLM + executing tests before exposing; if no solution passes, regenerate.
E15	AI-generated test cases wrong	False negatives	Mutation testing on tests; cross-validate vs reference solution; dedupe equivalent tests.
E16	API limits / cost spikes	Pipeline stalls	Rate-limit + queue (Celery); token budgets per curriculum; cache; fallback open-source models (Hybrid strategy).
E17	Latency during session	Bad UX	Pre-generate entire curriculum (async); session-time calls limited to evaluation (fast) + rare regeneration. Target <2s p95 eval.
E18	Concurrency (many users)	System overload	Stateless API workers; horizontal autoscale; GPU pool for ASR/OCR; queue backpressure.
E19	Sandbox code-execution security	RCE risk	judge0 (MVP) with resource limits; Firecracker microVM (prod) strong isolation; no filesystem persistence; CPU/mem/time/net caps.
E20	Offline / low bandwidth	Player unusable	Preload segments; offline exercise cache; PWA. (Stretch)
E21	Inappropriate/garbage video submitted	Wasted compute	Pre-filter: validate length, transcript language, topic relevance via embeddings before full pipeline; per-tenant quota.
E22	Duplicate ingestion	Cost	Content hash (video ID + transcript hash) → dedupe; reuse existing curriculum.
E23	Private / age-restricted / deleted YT URL	Ingestion fail	yt-dlp error handling; clear user-facing error + retry guidance; for uploads, validate codec/length.
E24	Uploaded file corrupt / bad codec	Pipeline crash	ffprobe validation pre-pipeline; reject early with explicit message.
E25	Cross-tenant data leakage	Security/privacy	Postgres Row-Level Security keyed by <code>tenant_id</code> ; per-tenant storage prefixes; sandbox tenant tagging; audit logs.
E26	Model drift / stale prompts	Quality decay	Versioned prompts; benchmark regression on golden set per release; drift alerts.
E27	Learner churns after repeated failures	UX	Soft-fail: after N fails, reveal explanation + offer “watch again” + simpler analog; never hard-block progression >1 level.

4 System Architecture & Module Breakdown

4.1 High-Level Architecture (text diagram)



Data flow summary: Video → Ingestion → (ASR + Vision) → Multimodal fusion → Concept segmentation → Checkpoint placement → Exercise + test generation (validated) → Persist curriculum JSON → Frontend plays → Session → Evaluation → Adaptive controller → Progress store → Dashboard.

Multi-tenancy note: Every record carries `tenant_id`; Postgres RLS policies enforce isolation; MinIO buckets/prefixes are tenant-scoped; judge0 submissions tagged with tenant; Celery tasks carry tenant context.

4.2 Module Specifications

4.2.1 M1 · Ingestion Pipeline — (Lead: Zubair; Support: Ahmed)

- **Input:** YouTube URL or uploaded file (+ `tenant_id`).
- **Output:** Local video file, audio track (16 kHz mono WAV), sampled frames (1 fps + shot-change frames), metadata (duration, language hint).
- **Tech:** `yt-dlp` (download, handles private/age-restricted errors gracefully), `ffmpeg/ffprobe` (demux, audio extract, validate codec, frame sample), `PySceneDetect` (shot boundaries), `OpenCV` (frame sampling + resize for OCR).
- **Papers/refs:** `yt-dlp` & `ffmpeg` docs; `PySceneDetect` docs.
- **Interface:** Emits `artifact_path` + manifest to Celery queue.

- **Validation:** Reject files >4 h, <30 s; reject unsupported codecs; dedupe by content hash.

4.2.2 M2 · Transcript & Metadata Extraction — (Lead: Aryan; Support: Zubair)

- **Input:** Audio track.
- **Output:** Timestamped transcript (word-level + segment-level), detected language, speaker labels, confidence.
- **Tech:** **Whisper large-v3** via **faster-whisper** (CTranslate2, $\sim 4\times$ faster, GPU-friendly, open-source, SOTA robust ASR, multilingual); **pyannote-audio 3.x** for diarization; **silero-vad** for VAD.
- **Papers:** “Robust Speech Recognition via Large-Scale Weak Supervision” (Whisper, Radford 2023) — **[MUST]**.
- **Interface:** JSON `{segments: [{start, end, text, words, speaker}], language, confidence}`.

4.2.3 M3 · Visual Content Extraction — (Lead: Ahmed; Support: Aryan)

- **Input:** Sampled/shot frames.
- **Output:** OCR’d code blocks (with timestamps), detected slides, diagrams, UI regions, keyframe set + per-item confidence.
- **Tech:** **PaddleOCR 2.7** (fast, multilingual, strong on code/syntax colors) primary; **TrOCR** (transformer-based, robust on degraded text) fallback; **Tesseract** baseline; **CLIP** for keyframe dedup/similarity; **DocLayNet** / **LayoutLMv3** for region detection (code vs slide vs diagram); optional **Real-ESRGAN** upscaling pre-OCR; post-process with **tree-sitter** to validate/sanitize extracted Python code (detect lang, parse, fix common OCR errors).
- **Papers:** CodeSCAN (programming screencast OCR) — **[MUST]**; “TrOCR” (Li 2022) — **[MUST]**; “Screen2Words” — **[OPT]**; “DocLayNet” — **[OPT]**; Real-ESRGAN — **[OPT]**.
- **Interface:** JSON `{frame_idx, ts, type, text, bbox, code_lang, confidence}`.

4.2.4 M4 · Lesson Structure Analyzer — (Lead: Aryan; Support: Ahmed)

- **Input:** Transcript segments + visual cues (slide changes, OCR topic words).
- **Output:** Ordered segments `[{start, end, title, summary, concepts[]}]` + structuredness score.
- **Tech:** Hybrid — TextTiling / BERT semantic embedding shift for candidate boundaries; **BERTopic** for topic modeling per chunk; **GPT-4o** (fallback Llama 3.1) for refinement + titles + summaries; BGE-M3 embeddings.
- **Papers:** “TextTiling” (Hearst 1997) — **[OPT]**; BERTopic (Grootendorst 2022) — **[MUST]**; How2 & Multimodal-Textbook — **[MUST]**(inspiration).
- **Interface:** `segments[]` schema.

4.2.5 M5 · Knowledge Graph / Concept Mapper — (Lead: Aryan; Support: Zubair)

- **Input:** Concepts from M4.
- **Output:** Concept nodes + prerequisite/dependency edges, mapped to a curated CS/programming concept taxonomy.
- **Tech:** LLM-extracted relations; stored as property graph (Postgres + adjacency, optional Neo4j later); link to Wikidata/CS taxonomy for canonical IDs.
- **Papers:** Concept-map learning literature; “Open Learner Models” — **[OPT]**.
- **Interface:** `graph: {nodes, edges}`.

4.2.6 M6 · Checkpoint Placement Controller — (Lead: Aryan; Support: Zubair)

- **Input:** Segments + concept graph + difficulty.
- **Output:** Ordered checkpoints [{ts, segment_id, concept_id, exercise_types[], difficulty}].
- **Logic:** Place checkpoint at topic transitions + after each “learnable” concept; density cap (≥ 90 s apart); avoid final 30 s; one exercise type per checkpoint (varied across curriculum).
- **Interface:** Consumed by frontend; cadence configurable per tenant.

4.2.7 M7 · Exercise Generation Engine — (Lead: Aryan; Support: Ahmed for code-context)

- **Input:** Segment + concept + extracted instructor code + difficulty + language (Python MVP).
- **Output:** Exercises of 4 types: MCQ (with distractors), coding challenge (prompt + starter + hidden tests + reference solution), debugging task (buggy snippet + tests), conceptual question (rubric + reference answer).
- **Tech:** GPT-4o primary generator (structured output via function calling / JSON schema); Qwen2.5-Coder-32B / DeepSeek-Coder-V2 for code-heavy generation & fallback; few-shot + concept-conditioned prompts; StarCoder2 / CodeT5 embeddings for similarity to detect copying instructor’s example (forces “new context”).
- **Papers:** “Learning to Generate Questions by Learning to Answer” — [OPT]; UniLM — [OPT]; APPS/HumanEval/MBPP for calibration — [MUST]; “Codex” (Chen 2021) — [OPT]; AlphaCode — [OPT].
- **Interface:** exercise JSON schema (shared contract, Appendix B).

4.2.8 M8 · Test Generation & Validation — (Lead: Aryan; Support: Zubair for sandbox)

- **Input:** Coding challenge prompt + reference solution.
- **Output:** Validated test cases (visible + hidden) + verified solvability.
- **Tech:** LLM generates N candidate tests + reference solution; **mutation testing** to ensure tests catch bugs; **CodeT** self-consistency (generate multiple solutions, keep tests passing on majority); execute in sandbox before publish.
- **Papers:** “CodeT: Code Generation with Generated Tests” (Chen 2022) — [MUST]; “Self-Debugging” (Chen 2023) — [OPT].
- **Interface:** tests[] schema; must pass sandbox before publish.

4.2.9 M9 · Answer Evaluation & Feedback Engine — (Lead: Aryan; Support: Zubair)

- **Input:** Learner response + exercise + (for code) test results.
- **Output:** Verdict, score, explanation, hints, code-quality notes, anti-cheat flag.
- **Tech:** MCQ $\rightarrow$ exact match + distractor analytics; code $\rightarrow$ judge0 tests + static analysis (ruff for Python MVP) + **LLM-as-a-Judge** with rubric for partial credit & style + **CodeBLEU** for similarity (anti-cheat); conceptual $\rightarrow$ LLM judge vs reference answer + embedding similarity threshold.
- **Papers:** “LLM-as-a-Judge” (Zheng 2023) — [MUST]; CodeBLEU — [OPT].
- **Interface:** eval_result JSON.

4.2.10 M10 · Adaptive Progression Controller — (Lead: Aryan; Support: Zubair)

- **Input:** Learner performance stream, concept graph, current difficulty.
- **Output:** Next checkpoint difficulty, remediation inserts, skip decisions.

- **Tech (MVP):** IRT 3PL (simpler, interpretable) or heuristic moving average. (**Phase 6 stretch**): Deep Knowledge Tracing (DKT) / DKVMN.
- **Papers:** DKT (Piech 2015) — **[MUST]**; DKVMN (Zhang 2017) — **[OPT]**; IRT — **[OPT]**.
- **Interface:** adaptive_state per learner per curriculum.

4.2.11 M11 · Learner Profile & Progress Tracker — (Lead: Zubair; Support: Aryan)

- **Input:** Session events, eval results.
- **Output:** Skill model, mastery per concept, history, spaced-repetition schedule (stretch).
- **Tech:** Postgres tables (RLS); **FSRS-4.5** for SRS (stretch); Redis for hot session state.
- **Papers:** FSRS — **[OPT]**; SM-2 — **[OPT]**.
- **Interface:** REST endpoints; WebSocket for live updates.

4.2.12 M12 · Frontend Application — (Lead: Zubair; Support: Aryan for eval data shapes)

- **Input:** Curriculum JSON + session API.
- **Output:** Interactive player UI.
- **Tech:** Next.js 14 (App Router) + TypeScript; **Monaco Editor** for code; **Plyr**/video.js or custom HTML5 player with checkpoint markers; Tailwind; Zustand for state; WebSocket for live eval; code-runner UX with stdout diff.
- **Interface:** Consumes REST + WS; renders exercise schema.

4.2.13 M13 · Backend API & Database — (Lead: Zubair)

- **Input/Output:** All REST + persistence.
- **Tech:** **FastAPI** (Python 3.11, async, typed — aligns with AI stack); **PostgreSQL 16** + **pgvector** (embeddings + relational, RLS); **Redis 7** (cache + queue broker); **Celery** (async pipeline); **MinIO/S3** (video/frame/artifact storage, tenant-scoped prefixes); **Auth** via OAuth (Google/GitHub) + JWT, per-tenant.
- **Papers:** “Designing Data-Intensive Applications” (Kleppmann) — **[MUST]**(Zubair).
- **Interface:** OpenAPI 3.1 spec (Appendix B).

4.2.14 M14 · Code Execution Sandbox — (Lead: Zubair; Support: Aryan)

- **Input:** Learner code + tests + language (Python MVP).
- **Output:** Pass/fail, stdout/stderr, runtime, exit code.
- **Tech (MVP):** **judge0** (Docker-based, multi-language, quick) with CPU/mem/time/net limits. (**Prod**): **Firecracker microVM** strongest isolation; nsjail fallback.
- **Papers:** Firecracker whitepaper — **[OPT]**(Zubair).

4.2.15 M15 · Evaluation & Monitoring Dashboard — (Lead: Zubair; Support: Aryan for metrics)

- **Input:** Pipeline logs, confidence scores, user feedback.
- **Output:** Quality metrics, drift, cost.
- **Tech:** Grafana + Prometheus; Sentry (errors); custom admin panel; per-item confidence logging; cost dashboards.

5 Team Role Definition & Ownership

5.1 Responsibility Matrix (RACI)

R = Responsible · A = Accountable · C = Consulted · I = Informed.

Module / Task	Zubair	Aryan	Ahmed
M1 Ingestion	A/R	C	C
M2 ASR / Transcript	I	A/R	C
M3 Visual Extraction (OCR/slides)	I	C	A/R
M4 Lesson Structure	C	A/R	C
M5 Concept Graph	C	A/R	I
M6 Checkpoint Placement	C	A/R	I
M7 Exercise Generation	C	A/R	C
M8 Test Gen & Validation	C	A/R	I
M9 Evaluation Engine	C (sandbox)	A/R	I
M10 Adaptive Controller	C	A/R	I
M11 Progress Tracker	A/R	C	I
M12 Frontend	A/R	C	I
M13 Backend/API/DB	A/R	C	I
M14 Code Sandbox	A/R	C	I
M15 Monitoring Dashboard	A/R	C	I
Prompt Library & Model Cards	I	A/R	C
Deployment / CI-CD / Infra	A/R	C	C
Eval / benchmark suite	C	A/R	C
Hardware companion (stretch)	I	C	A/R

5.2 Extent of Involvement

5.2.1 Zubair (Full-stack / System Glue) — owns product surface & reliability

Full ownership of: frontend (Next.js interactive player, exercise UI, dashboards), backend (FastAPI services, REST + WebSocket), data layer (Postgres/pgvector/Redis/MinIO with multi-tenant RLS), async orchestration (Celery), code execution sandbox (judge0 → Firecracker), auth & user management, progress storage, deployment (Docker, CI/CD, cloud), integration glue between all AI services, and the API contracts (co-authored with Aryan). He treats the AI modules as black-box services with defined I/O and ensures latency, caching, error handling, and a seamless UX.

Deliverables: working web app, OpenAPI spec, infra-as-code, CI/CD, monitoring, per-tenant isolation.

5.2.2 Aryan (AI Lead) — owns the intelligence & correctness

Full ownership of all AI/NLP/CV-*logic*: transcript cleaning & segmentation, concept extraction, checkpoint placement, exercise generation (MCQ/coding/debug/conceptual), test-case generation & validation, evaluation logic, adaptive difficulty, prompt engineering, model selection, model cards, and the evaluation/benchmark suite. Defines and publishes the **AI service API contracts** (request/response JSON schemas) Zubair consumes. Leads model evaluation, hallucination controls, and fine-tuning strategy if needed. Works with Ahmed to fuse visual + textual data.

Deliverables: AI pipeline services, prompt library (versioned), model cards, benchmark reports, AI API specs.

5.2.3 Muhammad Ahmed (CV/Hardware) — owns perception & multimodal fusion input

Primary owner of vision-heavy extraction: OCR pipeline for code (leveraging CodeSCAN insights), slide/diagram detection, keyframe selection, region classification, and confidence scoring. Handles

hardware-accelerated inference (GPU/CUDA, optional TensorRT) and latency/real-time constraints. Explores on-device / edge inference and the optional ESP32 e-ink companion device. Fuses visual data with Aryan's textual pipeline.

Deliverables: vision extraction service, OCR quality report, keyframe dataset curation, optional hardware prototype.

5.3 Interface Contracts (must be agreed before coding)

5.3.1 Shared data formats (canonical JSON)

```
// Transcript segment (Aryan -> all)
{ "id": int, "start": float, "end": float, "text": str,
  "speaker": str|null, "words": [{"w":str,"t":float}], "confidence": float }

// Visual extraction item (Ahmed -> Aryan)
{ "frame_idx": int, "ts": float, "type": "code|slide|diagram|ui",
  "text": str, "bbox": [x,y,w,h], "code_lang": str|null, "confidence": float }

// Concept (Aryan -> Zubair)
{ "id": str, "label": str, "description": str, "embedding": [float],
  "difficulty": 1..5 }

// Segment / topic (Aryan -> Zubair)
{ "id": str, "start": float, "end": float, "title": str, "summary": str,
  "concepts": [concept_id], "source_frames": [int] }

// Exercise (union schema by type) (Aryan -> Zubair)
{ "id": str, "type": "mcq|coding|debug|conceptual", "ts": float,
  "concept_id": str, "difficulty": 1..5,
  "prompt": str, "context": str|null,
  "mcq": {"options": [str], "answer_idx": int, "distractor_tags": [str]},
  "coding": {"starter": str, "tests_visible": [str], "tests_hidden": [str],
    "reference_solution": str, "language": "python", "constraints": [str]},
  "debug": {"buggy_code": str, "tests": [str], "bug_explanation": str},
  "conceptual": {"reference_answer": str, "rubric": [str], "min_similarity": float},
  "confidence": float, "validation_passed": bool }

// Eval result (Aryan -> Zubair/frontend)
{ "exercise_id": str, "verdict": "pass|fail|partial", "score": 0..1,
  "explanation": str, "hints": [str], "anti_cheat_flag": bool }
```

5.3.2 AI service API surface (Aryan owns, Zubair consumes)

- POST /ai/curriculum/generate {video_ref} → {curriculum_id} (async)
- GET /ai/curriculum/{id} → full curriculum JSON
- POST /ai/evaluate {exercise_id, response} → eval_result
- POST /ai/regenerate {exercise_id, constraints} → new exercise
- GET /ai/adaptive/{session_id} → next checkpoint + difficulty
- (Internal) POST /vision/extract {video_ref} → visual items (Ahmed)
- (Internal) POST /nlp/segment {transcript, visuals} → segments (Aryan)

5.3.3 Storage schema (Zubair owns)

Tables (ER in Appendix A): tenants, users, curricula, segments, concepts, concept_edges, exercises, tests, sessions, session_events, eval_results, skill_model, artifacts, prompt_versions. All tenant-scoped via tenant_id + RLS.

6 Technology Stack & Model Requirements

6.1 AI / Model Layer — (Aryan + Ahmed)

Component	Primary Model/Lib	Why	Fallback
ASR	Whisper large-v3 via faster-whisper (CTranslate2)	SOTA robust, multilingual, word timestamps, open-source	Whisper medium-v2; OpenAI Whisper API
Diarization	pyannote-audio 3.x	Best open-source speaker segmentation	None (single-speaker assumed)
VAD	silero-vad	Fast, accurate silence detection	WebRTC VAD
OCR (code)	PaddleOCR 2.7	Fast, multilingual, strong on code/syntax colors	TrOCR, Tesseract
OCR (degraded) Frame/keyframe	TrOCR (Microsoft) OpenCV + PySceneDetect + CLIP (dedup)	Transformer, robust to low-res Mature + semantic similarity	easyocr —
Layout/region	LayoutLMv3 / DocLayNet	Code-vs-slide-vs-diagram classification	Rule-based heuristics
Upscaling	Real-ESRGAN	Restore low-res code frames	None
Embeddings	BGE-M3 (multilingual) or text-embedding-3-large	SOTA retrieval, open option	sentence-transformers MPNet
Topic modeling	BERTopic	Neural topic clustering, dynamic	LDA
Segmentation	Embedding-shift + TextTiling + LLM refine	Hybrid robustness	Pure LLM chunking
Summarization	GPT-4o / BART-large-CNN (bulk)	Quality vs cost trade	T5-large
Generation (general)	GPT-4o	Best instruction-following, structured output	Llama 3.1 70B (Groq/Together)
Code gen/grader	GPT-4o + Qwen2.5-Coder-32B (fallback/bulk)	Strong code, open-source sovereignty	DeepSeek-Coder-V2, StarCoder2
Code similarity	CodeBLEU + AST diff (tree-sitter)	Semantic+structural	Jaccard token overlap
Static analysis	ruff (Python MVP)	Fast modern linter	pylint
LLM-as-judge	GPT-4o with rubric	Reliable grading	Llama-3.1-70B judge
Knowledge tracing	IRT 3PL (MVP) → DKT (stretch)	Interpretable now, deep later	Heuristic moving average
SRS (stretch)	FSRS-4.5	Modern, accurate spacing	SM-2
Code execution	judge0 (MVP) → Firecracker microVM (prod)	Isolation + speed	Docker+nsjail

Justification summary: Hybrid open-source + proprietary gives cost control, sovereignty, and a clear degradation path if GPT-4o is unavailable/expensive. Whisper/PaddleOCR run on a self-hosted GPU to avoid per-minute API costs. GPT-4o is reserved for high-value generation; bulk operations use Llama 3.1 / Qwen-Coder.

6.2 Application / Infra Layer — (Zubair)

Layer	Choice	Why
Backend	FastAPI (Python 3.11)	Async, typed, shares language with AI stack
Frontend	Next.js 14 (App Router) + TS + Tailwind	SSR, fast, mature ecosystem
Code editor	Monaco Editor	VS Code-grade UX
Video player	Custom HTML5 + Plyr /video.js	Checkpoint overlay control
DB	PostgreSQL 16 + pgvector	Relational + vector in one; RLS for tenancy
Cache/Queue	Redis 7 + Celery	Task orchestration
Object storage	MinIO (self-host) / S3	Video, frames, artifacts; tenant-scoped prefixes
Vector DB	pgvector (sufficient at MVP scale) → Qdrant if needed	Avoid extra infra
Realtime	WebSocket / Server-Sent Events	Live eval + session
Container	Docker + Docker Compose (dev) → Kubernetes (prod)	Portability
CI/CD	GitHub Actions	Standard
IaC	Terraform (Phase 5)	Reproducible infra

continued on next page

Layer	Choice	Why
Monitoring	Prometheus + Grafana + Sentry	Metrics + errors
Auth	OAuth (Google/GitHub) + JWT, per-tenant	Low-friction onboarding
Code sandbox	judge0 (MVP) → Firecracker (prod)	Per locked decision

6.3 Hardware / Infrastructure

- **Dev:** 1× GPU node (e.g., 1× A10g 24 GB or RTX 4090) for Whisper + OCR + open LLMs; CPU nodes for FastAPI/Celery.
- **Prod (MVP):** GPU instance for batch ASR/OCR; CPU autoscale for API; managed Postgres; judge0 pool.
- **RAM:** ≥ 32 GB on AI node; 16 GB on API nodes.
- **Storage:** ≥ 500 GB object storage for videos/frames.
- **Cost knobs:** token budget per curriculum; cache transcripts/embeddings; tier models; dedupe identical videos; per-tenant quotas.

6.4 Fallback / Degradation Strategy

Trigger	Fallback
GPT-4o API down or over budget	Llama 3.1 70B (Groq/Together) for generation; Qwen2.5-Coder for code
OCR confidence low	Transcript-only exercises; flag “code unclear”
ASR WER high	Slide + concept-only mode; allow manual transcript upload
judge0 pool saturated	Queue submissions; return “grading pending” + web-hook
GPU node down	Cloud Whisper API + cloud OCR (higher cost, capped)
Per-tenant budget exceeded	Pause pipeline; notify admin; allow top-up

7 Pre-Implementation Research & Reading List

Read *before* coding the relevant module. Tag: **[MUST]** = Must read · **[OPT]** = Optional deep dive. Owner in parentheses.

7.1 NLP / Transcript Understanding — (Aryan)

- **[MUST]** “Robust Speech Recognition via Large-Scale Weak Supervision” (Whisper, Radford et al., 2023)
- **[MUST]** How2 Dataset paper (Narasimhan et al., 2018) — multimodal instructional + summaries
- **[MUST]** HowTo100M (Miech et al., 2019) — scale of instructional video
- **[MUST]** Multimodal-Textbook / Multimodal-Textbook-6.5M — keyframe + OCR + ASR
- **[MUST]** BERTopic (Grootendorst, 2022) — topic modeling
- **[OPT]** TextTiling (Hearst, 1997); text-segmentation surveys
- **[OPT]** BART (Lewis 2020) / T5 (Raffel 2020) for summarization
- **[OPT]** pyannote-audio & silero-vad docs

7.2 Question Generation — (Aryan)

- **[MUST]** Educational Question Generation surveys (Kurdi et al., 2020)
- **[MUST]** APPS / HumanEval / MBPP benchmarks (calibrate coding difficulty)
- **[OPT]** “Learning to Generate Questions by Learning to Answer” (Chan & Fan)
- **[OPT]** UniLM (Dong et al., 2019); ProphetNet
- **[OPT]** Distractor generation with transformers
- **[MUST]** Prompt-engineering best practices (OpenAI cookbook, structured outputs)

7.3 Code Generation & Evaluation — (Aryan)

- **[MUST]** “CodeT: Code Generation with Generated Tests” (Chen et al., 2022)
- **[MUST]** “LLM-as-a-Judge” (Zheng et al., 2023)
- **[OPT]** “Codex” (Chen et al., 2021); AlphaCode (Li et al., 2022)
- **[OPT]** “Self-Debugging” (Chen et al., 2023); “Self-Edit”
- **[OPT]** CodeBLEU (Ren et al., 2020)
- **[OPT]** StarCoder2 / DeepSeek-Coder / Qwen2.5-Coder technical reports
- **[MUST]** tree-sitter docs (AST parsing, language detection)

7.4 Vision / OCR / Screen Understanding — (Ahmed)

- **[MUST]** CodeSCAN dataset & paper (programming screencast OCR)
- **[MUST]** “TrOCR: Transformer-based Optical Character Recognition” (Li et al., 2022)
- **[MUST]** PaddleOCR docs & benchmarks
- **[OPT]** “Screen2Words” / screen summarization
- **[OPT]** DocLayNet (layout); LayoutLMv3
- **[OPT]** Real-ESRGAN (image restoration) paper
- **[OPT]** “SlideNet” / slide-classification literature
- **[MUST]** CLIP for keyframe similarity

7.5 Multimodal / Video-Language — (Aryan + Ahmed)

- **[OPT]** VideoBERT (Sun et al., 2019)
- **[OPT]** VIOLET (Fu et al., 2021)
- **[OPT]** InternVideo (Wang et al., 2024)
- **[OPT]** Video-LLaVA / VideoChat (for optional VQA)
- **[OPT]** TimeSformer (Bertasius et al., 2021)

7.6 Adaptive Learning — (Aryan)

- **[MUST]** “Deep Knowledge Tracing” (Piech et al., 2015)
- **[OPT]** DKVMN (Zhang et al., 2017)
- **[OPT]** Bayesian Knowledge Tracing primer; Item Response Theory (3PL)
- **[OPT]** FSRS whitepaper; SM-2 algorithm
- **[OPT]** “Open Learner Models” survey

7.7 System Design / Infra — (Zubair)

- **[MUST]** “Designing Data-Intensive Applications” (Kleppmann) — batching, streaming, queues
- **[MUST]** Celery + Redis patterns; task idempotency
- **[MUST]** FastAPI async + background tasks docs
- **[OPT]** Firecracker microVM whitepaper; judge0 docs
- **[OPT]** WebSocket vs SSE trade-offs; idempotency keys
- **[MUST]** Docker/Compose + GitHub Actions CI patterns
- **[OPT]** Securing code-execution sandboxes (nsjail/gVisor)
- **[MUST]** Postgres Row-Level Security patterns (multi-tenancy)

8 Development Phases & Milestones

8.1 Phase 0 · R&D Spike & Foundations — (Week 1) — All

- Set up monorepo, Docker Compose, CI, env.
- Each member reads **[MUST]** items for their domain.
- Stand up Whisper + PaddleOCR on GPU node; confirm a sample tutorial runs end-to-end manually.
- **Lock API contracts (§5.3) by end of week.**
- **Acceptance:** Each can run a demo notebook of their core model on a sample tutorial; contracts signed off.

8.2 Phase 1 · Ingestion + Transcript + Vision Extraction — (Weeks 2–3)

Zubair (M1, infra), Aryan (M2), Ahmed (M3)

- Implement video download/upload, audio extraction, frame sampling.
- Whisper transcript service with timestamps (Aryan).
- OCR + slide/keyframe pipeline (Ahmed).
- Persist artifacts to MinIO + Postgres (tenant-scoped).
- **Acceptance:** Given a URL/upload, system produces transcript JSON + visual-items JSON stored and retrievable via API.

8.3 Phase 2 · Concept Segmentation + Knowledge Graph + Checkpoints — (Weeks 4–5)

Aryan (M4–M6), Zubair (storage/API), Ahmed (visual fusion)

- Segment transcript into topics (hybrid); extract concepts; build concept graph; place checkpoints.
- Wire to DB; expose `/ai/curriculum/generate` + `/ai/curriculum/{id}`.
- **Acceptance:** A curriculum JSON with segments, concepts, and checkpoint plan viewable in a raw admin page; $\geq 80\%$ of checkpoints land on real topic boundaries on the 5-video golden test set.

8.4 Phase 3 · Exercise Generation + Test Validation + Evaluation — (Weeks 6–8)

Aryan (M7–M9), Zubair (sandbox M14 + API), Ahmed (code-context)

- Generate MCQ/coding/debug/conceptual per checkpoint; generate + validate tests; anti-cheat.
- Code sandbox (judge0) + static analysis (ruff) + LLM-judge.
- **Acceptance:** For a test video, $\geq 90\%$ of generated coding challenges pass automated validation (solvable by reference solution); eval engine returns correct verdicts on held-out set ($\geq 85\%$ agreement with human rubric).

8.5 Phase 4 · Interactive Frontend + Adaptive Logic + Progress — (Weeks 9–11)

Zubair (M12, M11), Aryan (M10)

- Interactive video player w/ checkpoint overlays; Monaco code runner; results UI.
- Adaptive controller (IRT 3PL); progress dashboard; session state.

- **Acceptance:** End-to-end user flow: submit video → wait → play → hit checkpoint → solve → get feedback → adapt → progress updates. p95 eval latency <2s.

8.6 Phase 5 · Polish, Evaluation, Deployment — (Weeks 12–13) — All

- Benchmark suite (segmentation precision/recall, exercise quality via human ratings, anti-cheat FP rate, cost/curriculum, latency percentiles).
- Monitoring dashboard; cost controls; error handling & retries; RLS audit.
- Deploy to staging → production; user-test with 5 real learners.
- **Acceptance:** 3 videos ingested end-to-end on prod; $\geq 4/5$ testers rate experience “useful”; p95 eval latency <2s; daily cost bounded.

8.7 Phase 6 · Stretch Features — (post-MVP) — All

- Flashcards, notes, concept graph viz, SRS, interview mode, voice checkpoints, hardware companion (Ahmed), JS/TS languages.

9 Documentation & Guidelines Blueprint

Alongside code, the team **must** produce:

1. **System Design Document** — this plan, refined after Phase 0; living doc.
2. **API Contracts** — OpenAPI 3.1 specs for every endpoint (Zubair + Aryan co-author); auto-generated client types for frontend.
3. **Data Schema Design** — ER diagram (Appendix A) for all Postgres tables + pgvector usage.
4. **Model Cards** (Aryan) — per AI component: model name/version, input/output, training/few-shot data, limitations, latency, cost, fallback.
5. **Prompt Library** (Aryan) — version-controlled prompts (`prompts/v1/*.txt` + metadata YAML: model, temperature, eval score); A/B testable. Spec in Appendix C.
6. **Evaluation Reports** — segmentation precision/recall, exercise quality (human-rated rubric), anti-cheat FP rate, cost/curriculum, latency percentiles.
7. **Vision Quality Report** (Ahmed) — OCR WER on CodeSCAN-style samples, per-frame-type confidence stats.
8. **User Onboarding Guide** — how a learner submits a video & uses sessions.
9. **Developer Onboarding Guide** — repo layout, env setup, how to add an AI service, how to add an exercise type, deployment.
10. **Runbook** — incident response, known failures, retry/idempotency notes.
11. **Risk & Decision Log** — ADRs (Architecture Decision Records) for every major choice.
12. **Security & Tenancy Doc** — RLS policies, sandbox isolation, per-tenant quotas, audit logging.

10 Innovation & Creative Ideas

1. **“Reverse Turing Test” exercises** — system shows an AI-generated code snippet with a subtle bug; learner must spot it. Tests critical reading & doubles as hallucination training data for Aryan’s eval suite.
2. **Concept dependency graph as a learnable roadmap** — interactive node-link viz; learner sees prerequisites & unlocks; clicking reveals mastery %; auto-suggests “next concept to learn.”
3. **Peer-review simulation** — system generates a PR (code + diff) and asks learner to review like a senior dev (find bugs, suggest improvements); LLM grades the review quality. Fills the “code review” skill gap self-taught devs lack.
4. **Voice-interactive checkpoints** — TTS asks a question, learner answers by voice (STT); great for conceptual recall & accessibility; enables hands-busy learning.
5. **Hardware companion device (Ahmed)** — ESP32 with e-ink display shows the current concept name, hint, or focus timer; optional “focus sensor” (PIR/camera blink detection) that pauses video when learner looks away.
6. **Adaptive “second chance” regeneration** — on failure, generate a *simpler* analog of the same concept (not the same question easier) so the learner practices transfer again, not memorization.
7. **Knowledge-provenance overlay** — every generated exercise links to the exact transcript timestamp + OCR frame it was derived from (transparency + trust + audit).
8. **“Tutorial Hell” score** — a metric that predicts if a learner is passively copying (high code similarity to instructor, fast submissions, low explanation quality) and nudges them toward independent practice.
9. **Cross-video concept linking** — when a learner struggles on a concept, recommend a *different* ingested video’s segment on the same concept (curriculum-as-a-graph across videos).
10. **Difficulty-aware test amplification** — generate *adversarial* test cases (edge inputs) tuned to the learner’s weak spots for extra stretch.

11 Final Checklist Before Implementation Start

11.1 R&D

- ☐ Each member has read all **[MUST]** items for their domain (§7).
- ☐ Whisper large-v3 runs on a sample tutorial with acceptable WER measured.
- ☐ PaddleOCR + TrOCR tested on IDE screenshots; WER on code measured (Ahmed).
- ☐ GPT-4o structured-output prompt for exercise generation prototyped & sanity-checked (Aryan).
- ☐ Open-source fallback (Qwen2.5-Coder / Llama 3.1) accessible via Together/Groq or local.
- ☐ Datasets obtained: CodeSCAN sample, How2/HowTo100M sample, APPS/HumanEval/MBPP for calibration.

11.2 Environments & Access

- ☐ Monorepo created; Docker Compose dev env boots (Postgres+pgvector, Redis, MinIO, Celery, judge0).
- ☐ GPU node provisioned; CUDA + PyTorch + faster-whisper + PaddleOCR verified.
- ☐ API keys secured in secret manager (OpenAI, etc.); spending caps set.
- ☐ GitHub Actions CI green on a skeleton commit.

11.3 Contracts & Schemas

- ☐ API contracts (§5.3, Appendix B) reviewed & signed by all three.
- ☐ JSON schemas for transcript/visual/segment/exercise/eval_result committed + codegen.
- ☐ ER diagram + migrations drafted (Appendix A).
- ☐ Postgres RLS policy draft for multi-tenancy.

11.4 Quality gates

- ☐ Eval rubric for human rating of generated exercises defined (Aryan) (Appendix D).
- ☐ A 5-video golden test set selected for regression benchmarks.
- ☐ Cost budget per curriculum defined; token counters in place.

A Data Schema (ER)

```
tenants(id PK, name, slug, plan, created_at)
users(id PK, tenant_id FK->tenants, email, role[learner|instructor|admin],
      oauth_provider, created_at) -- RLS by tenant_id

curricula(id PK, tenant_id FK, source_type[youtube|upload], source_ref,
          content_hash UNIQUE, title, status[queued|processing|ready|failed],
          language, framework_version, duration, created_at, ready_at)

segments(id PK, curriculum_id FK, idx, start, end, title, summary, confidence)
concepts(id PK, tenant_id FK, label, description, embedding vector(1024),
         difficulty 1..5, canonical_id)
concept_edges(id PK, src_concept_id FK, dst_concept_id FK, type[prereq|related])

checkpoints(id PK, curriculum_id FK, ts, segment_id FK, concept_id FK,
            exercise_types[], difficulty, ordering)

exercises(id PK, checkpoint_id FK, type[mcq|coding|debug|conceptual], payload JSONB,
          confidence, validation_passed, prompt_version_id FK, created_at)

tests(id PK, exercise_id FK, kind[visible|hidden], input, expected, weight)

sessions(id PK, user_id FK, curriculum_id FK, started_at, completed_at, resume_ts)
session_events(id PK, session_id FK, ts,
              type[play|pause|checkpoint|submit|feedback], payload JSONB)

eval_results(id PK, session_event_id FK, exercise_id FK, verdict,
            score 0..1, explanation, anti_cheat_flag, created_at)

skill_model(id PK, user_id FK, concept_id FK, mastery 0..1, attempts, last_updated)
prompt_versions(id PK, name, version, model, temperature, eval_score, body, created_at)
artifacts(id PK, tenant_id FK, curriculum_id FK,
          kind[video|audio|frame|transcript|ocr], storage_uri, meta JSONB)
```

RLS: every tenant-scoped table has policy USING (tenant_id = current_setting('app.tenant_id')::uuid).
API sets app.tenant_id per request from JWT.

B AI Service API Contracts (OpenAPI sketch)

```
openapi: 3.1.0
info: { title: Curriculum AI API, version: 1.0.0 }
paths:
  /ai/curriculum/generate:
    post:
      summary: Create a curriculum from a video (async)
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                video_ref: { type: string } # URL or upload artifact id
                tenant_id: { type: string }
      responses:
        '202':
          description: Accepted
          content:
            application/json:
              schema:
                type: object
                properties:
                  curriculum_id: { type: string }
                  status: { type: string }
  /ai/curriculum/{id}:
    get:
      summary: Fetch full curriculum JSON
      parameters: [{ name: id, in: path, required: true, schema: { type: string } }]
      responses:
        '200':
          description: Curriculum
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Curriculum' }
        '202': { description: Still processing }
  /ai/evaluate:
    post:
      summary: Evaluate a learner response
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                exercise_id: { type: string }
                response: { type: string } # code or option idx or text
      responses:
        '200':
          description: Eval result
          content:
            application/json:
              schema: { $ref: '#/components/schemas/EvalResult' }
  /ai/regenerate:
    post:
      summary: Regenerate a fresh exercise variant
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                exercise_id: { type: string }
                constraints: { type: object }
      responses:
        '200':
          description: New exercise
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Exercise' }
  /ai/adaptive/{session_id}:
    get:
```

```

summary: Next checkpoint + difficulty
parameters: [{ name: session_id, in: path, required: true,
               schema: { type: string } }]
responses:
  '200':
    description: Adaptive state
    content:
      application/json:
        schema:
          type: object
          properties:
            next_checkpoint_id: { type: string }
            difficulty: { type: integer }
            remediate: { type: boolean }
components:
  schemas:
    Curriculum:
      type: object
      properties:
        id: { type: string }
        segments: { type: array, items: { type: object } }
        concepts: { type: array, items: { type: object } }
        checkpoints: { type: array, items: { type: object } }
    Exercise:
      type: object
      properties:
        id: { type: string }
        type: { type: string, enum: [mcq, coding, debug, conceptual] }
        ts: { type: number }
        concept_id: { type: string }
        difficulty: { type: integer }
        prompt: { type: string }
        confidence: { type: number }
        validation_passed: { type: boolean }
    EvalResult:
      type: object
      properties:
        exercise_id: { type: string }
        verdict: { type: string, enum: [pass, fail, partial] }
        score: { type: number }
        explanation: { type: string }
        hints: { type: array, items: { type: string } }
        anti_cheat_flag: { type: boolean }

```

C Prompt Library Spec

C.0.1 Directory layout:

```
prompts/  
  v1/  
    segment_topics.txt      # M4  
    extract_concepts.txt    # M5  
    place_checkpoints.txt   # M6  
    gen_mcq.txt             # M7  
    gen_coding.txt          # M7  
    gen_debug.txt           # M7  
    gen_conceptual.txt      # M7  
    gen_tests.txt           # M8  
    judge_code.txt          # M9 (LLM-as-judge)  
    judge_conceptual.txt    # M9  
    explain_concept.txt     # UC-12  
  manifest.yaml
```

C.0.2 manifest.yaml per prompt:

```
- name: gen_coding  
  version: v1  
  model: gpt-4o  
  temperature: 0.4  
  response_format: json_schema # structured output  
  schema_ref: schemas/exercise_coding.json  
  few_shot: 3  
  eval_score: 0.0 # updated by benchmark run  
  fallback_model: Qwen2.5-Coder-32B  
  notes: "Force new context vs instructor example via negative-constraint block."
```

Versioning rule: never edit a prompt in place; create v2/ + A/B against golden set; keep `eval_score` per version; link `prompt_version_id` on every generated exercise for traceability.

D Golden Test Set & Eval Rubrics

D.1 Golden test set (5 videos)

- Python basics tutorial (~15 min) — clear code on screen.
- FastAPI walkthrough (~25 min) — slides + code.
- Pandas tutorial (~40 min) — long, mixed content.
- Django REST (~30 min) — IDE-heavy.
- React-for-Python-devs (~20 min) — non-Python → tests MCQ-only fallback.

D.2 Segmentation rubric (Aryan)

- **Precision/Recall** of detected boundaries vs human-annotated boundaries (± 15 s tolerance).
- Target: $\geq 80\%$ F1 on golden set.

D.3 Exercise quality rubric (human-rated, Aryan)

Each generated exercise scored 1–5 on:

- **Correctness:** does the reference solution pass its own tests?
- **Solvability:** is it solvable by an independent solver LLM?
- **Relevance:** does it test the target concept?
- **Novelty:** is the context distinct from the instructor’s example? (anti-copy)
- **Clarity:** is the prompt unambiguous?
- Target: mean $\geq 3.5/5$; reject & regenerate if < 3 .

D.4 Anti-cheat FP rate (Aryan)

- On a held-out set of genuine independent solutions, measure false “cheating” flags.
- Target: $< 5\%$ FP.

D.5 Eval-engine agreement (Aryan)

- LLM-judge verdicts vs human rubric on held-out set.
- Target: $\geq 85\%$ agreement.

D.6 System metrics (Zubair)

- p95 eval latency < 2 s.
- p95 curriculum generation < 8 min for a 30-min video.
- Cost per curriculum $\leq$ defined budget.
- Pipeline success rate $\geq 95\%$ (no human intervention).

End of Master Plan v1.0.